

Ingeniería de datos y modelos de clasificación para detección de fraude en transacciones

Estudio comparativo sobre la competencia IEEE-CIS Fraud Detection
Proyecto 1 — Planificación

Edgar Chambilla

Septiembre de 2026

Resumen

Este informe documenta la reproducción local y el análisis comparativo de soluciones públicas de la competencia *IEEE-CIS Fraud Detection* (Kaggle, 2019). El estudio busca identificar qué decisiones de tratamiento de datos y modelado explican el rendimiento observado. Se seleccionaron 12 notebooks, se descargaron sus fuentes y se ejecutaron localmente los 4 que producen modelos entrenables, usando el conjunto completo y una validación temporal equivalente a la separación train/test de la competencia. La ablación principal mantiene fijo XGBoost, con profundidad 12: al añadir un pseudoidentificador de cliente (*magic UID*) y unas 40 agregaciones, el AUC local sube de 0,9375 a 0,9479 (+0,0104). El cambio de representación aporta más que cambiar de algoritmo o de hardware. Como extensión, se implementó un sistema adaptativo con features causales, ventanas expansivas y deslizantes, monitoreo PSI y costos de decisión. Los experimentos se pueden repetir con los scripts incluidos.

Índice general

1. Introducción	5
1.1. Motivación	5
1.2. Objetivos	5
1.3. Metodología general	5
1.4. Estructura del informe	6
2. Conjunto de datos	7
2.1. Origen y estructura	7
2.2. Desbalance del problema	7
2.3. Calidad de los datos	8
2.4. La estructura temporal: la dificultad central	8
3. Tratamiento de datos e ingeniería de variables	11
3.1. Integración y tipos de datos	11
3.2. Valores faltantes: codificar en vez de imputar	11
3.3. Codificación de variables categóricas	11
3.4. Normalización de columnas D (estabilización temporal)	12
3.5. Selección por consistencia temporal	12
3.6. El pseudo-identificador de cliente (<i>magic UID</i>)	13
3.7. Protocolo de validación	13
3.8. Limitación conocida del protocolo	14
4. Sistema adaptativo y concept drift	15
4.1. Features causales	15
4.2. Ventanas y detección de drift	15
4.3. Resultados de la evaluación adaptativa	15
4.4. Decisión y adaptación operativa	16
5. Modelos comparados	17
5.1. Baseline XGBoost (inversion, xhlulu)	17
5.2. LightGBM con selección de variables (nroman)	17
5.3. XGBoost con ingeniería de entidades (cdeotte XGB_95 / XGB_96)	17
5.4. Costo computacional	18
6. Resultados	19
6.1. Comparación principal	19
6.2. Ablación del magic UID	19
6.3. Qué variables dominan	20
6.4. Referencias cruzadas con Kaggle	20
7. Discusión	22
7.1. Ventajas y límites de cada método	22
7.2. Qué importa realmente: jerarquía de impacto	22
7.3. Limitaciones metodológicas	23
7.4. Amenazas a la validez	23

8. Conclusiones y propuesta	24
8.1. Conclusiones	24
8.2. Propuesta de desarrollo	24
A. Reproducibilidad	25
A.1. Estructura del proyecto	25
A.2. Entorno	25
A.3. Ejecución de los experimentos	25
A.4. Adaptaciones de versiones (2019 → 2026)	25
A.5. Protocolo ético	26

Índice de figuras

2.1. Distribución de la variable objetivo. El desbalance 1:27 exige métricas de orden (AUC) y hace innecesario el re-muestreo agresivo para modelos de árboles.	8
2.2. Porcentaje promedio de valores faltantes por familia de variables. La ausencia de la tabla de identidad es estructural (transacciones sin huella de dispositivo) y, como se verá, codificar la ausencia como categoría propia (-999) es preferible a imputar.	8
2.3. Distribución del monto por clase en escala logarítmica. Las transacciones fraudulentas se concentran en montos bajos-repetitivos, con una cola superior más densa que la de las legítimas.	9
2.4. Tasa diaria de fraude durante los 182 días de entrenamiento. Los picos agudos corresponden a campañas de ataque acotadas en el tiempo: el patrón no es estacionario, y por eso la selección de variables por <i>consistencia temporal</i> (Section 3.5) resulta decisiva.	9
2.5. Histograma del tablero público de la competencia. En rojo, los cinco primeros equipos: AlKo (0,9681), FraudSquad (0,9677), Young for you (0,9676), 2 uncles and 3 puppies (0,9672) y Mr Lonely (0,9663).	10
3.1. Normalización de D15. Izquierda: la variable cruda deriva con el tiempo. Derecha: tras restar el reloj absoluto queda una banda estable, comparable entre entrenamiento y futuro.	12
3.2. Esquema de la técnica del <i>magic UID</i> : reconstrucción del cliente a partir de (3.1) y generación de agregaciones por grupo. El identificador se usa solo como llave de agrupación y se descarta antes del entrenamiento.	13
4.1. Arquitectura propuesta: el monitoreo temporal alimenta la selección de ventana y el reentrenamiento del modelo.	15
4.2. Evolución de AUC, F1 y PSI por bloque. El descenso del segundo bloque muestra que el rendimiento puede degradarse aunque el drift distributivo no supere el umbral alto de PSI; por ello se monitorean también métricas de desempeño. . . .	16
6.1. AUC local por método. El salto relevante ocurre al pasar de features crudas a la ingeniería de entidades (rojo): el mismo algoritmo gana +0,018 con la preparación de datos de Deotte y +0,010 adicionales con el magic UID.	19
6.2. Precisión vs. costo computacional. La GPU reduce el tiempo del baseline de 112 s a 25 s (4.4×) sin alterar el AUC; la mayor precisión exige más cómputo (profundidad 12 y hasta 5 000 árboles) pero la ganancia dominante proviene de los datos, no del cómputo.	20
6.3. Ablación con/sin magic UID sobre el mismo modelo (XGBoost d12). La diferencia entre barras del mismo color es el aporte puro de la ingeniería de entidades. . . .	21
6.4. Importancia (gain) de las 20 variables principales en un XGBoost exploratorio. El peso de conteos, deltas temporales y tarjetas anticipa por qué las agregaciones por cliente resultan tan efectivas.	21

Índice de tablas

2.1. Resumen de los archivos del conjunto de datos.	7
5.1. Configuraciones comparadas. “d9/d12” denota profundidad máxima del árbol. . .	17
6.1. Ranking local: AUC y costo computacional por método. El AUC principal de cada método se reporta en su esquema de validación propio (columna Validación).	19
7.1. Ventajas y limitaciones observadas en la reproducción local.	22

Introducción

1.1 Motivación

El fraude en pagos *card-not-present* (CNP) genera pérdidas anuales de miles de millones de dólares y plantea un problema de clasificación binaria particular: el conjunto de datos está fuertemente desbalanceado (alrededor del 3,5 % de transacciones fraudulentas), y las features son en su mayoría anónimas o están ofuscadas por razones comerciales. El futuro tampoco se parece al pasado: los patrones de ataque cambian, por lo que un modelo que sobreajusta el período de entrenamiento degrada rápidamente su desempeño en producción.

La competencia *IEEE-CIS Fraud Detection* [2] (Kaggle, 2019), organizada por la IEEE Computational Intelligence Society junto a Vesta Corporation, formalizó este problema sobre transacciones reales de comercio electrónico. Contó con 6 355 equipos participantes y consolidó un cuerpo público de soluciones, entre las que destaca la técnica del *magic UID* de Chris Deotte y Konstantin Yakovlev (equipo *FraudSquad*, 2.º en el tablero público y 1.º en el privado).

1.2 Objetivos

1. **Recolectar** las mejores soluciones públicas de la competencia (notebooks con mayor reputación y write-ups de equipos ganadores).
2. **Reproducir localmente** los notebooks que producen modelos entrenables, mediendo AUC y costo computacional sobre hardware propio (16 núcleos CPU, NVIDIA RTX 2060 de 6 GB).
3. **Aislar** mediante ablación controlada qué componente explica la ganancia de rendimiento: el algoritmo, el hardware o la ingeniería de variables.
4. **Documentar** las técnicas de tratamiento de datos y de modelado, dejando un pipeline reproducible como base para una propuesta propia.

1.3 Metodología general

El estudio siguió cuatro fases:

1. **Selección de código:** se identificaron los 12 notebooks públicos con mayor reputación (2986 a 169 votos) mediante el índice curado *Must-see Kernels* [1] y el API público de Kaggle. Cada fuente se descargó en formato `.ipynb` y se exportó a Python legible.
2. **Triaje:** se clasificó cada notebook en *ejecutable* (produce un modelo entrenable) o *analítico* (EDA pura, técnicas de re-muestreo, infraestructura GPU no compatible con 6 GB de VRAM). Cuatro notebooks resultaron ejecutables.
3. **Reproducción local:** los 4 ejecutables se adaptaron a las versiones modernas de las librerías ($\text{XGBoost} \geq 2$, $\text{LightGBM} \geq 4$, $\text{pandas} 3$) y se ejecutaron con el conjunto completo de entrenamiento, bajo un protocolo de validación temporal uniforme (Section 3.7).
4. **Análisis comparativo:** se comparó AUC y tiempo de cómputo, y se realizó una ablación *con/sin magic UID* sobre el mismo modelo para atribuir la ganancia.

1.4 Estructura del informe

[Chapter 2](#) describe el conjunto de datos; [Chapter 3](#) detalla las técnicas de tratamiento y de ingeniería de variables; [Chapter 5](#) presenta los modelos comparados; [Chapter 6](#) muestra los resultados locales con sus gráficos; [Chapter 7](#) discute ventajas, límites metodológicos y lecciones; [Chapter 8](#) concluye y propone el desarrollo futuro. [Appendix A](#) documenta la reproducibilidad completa.

Conjunto de datos

2.1 Origen y estructura

Los datos provienen de transacciones reales de comercio electrónico provistas por Vesta Corporation. Se componen de dos tablas unidas por **TransactionID** (Table 2.1): una tabla de *transacción* (590 540 filas $\times$ 394 columnas en entrenamiento) y una tabla de *identidad* (144 233 filas $\times$ 40 columnas), disponible solo para el ~ 24 % de las transacciones.

Tabla 2.1: Resumen de los archivos del conjunto de datos.

Archivo / magnitud	Filas	Columnas	Nota
train_transaction.csv	590,540	393	transacciones + 339 V + conteos + tiempos
train_identity.csv	144,233	40	dispositivo / red (presente en 24 % de las filas)
test_transaction.csv	506,691	393	mes posterior al train (sin target)
Fraude (isFraud=1)	20,663	—	3.50 %

Las variables se agrupan en familias semánticas:

- **Montos y producto:** TransactionAmt, ProductCD (W, C, R, H, S).
- **Tarjeta:** card1–card6 (emisor, tipo, categoría).
- **Geografía:** addr1–addr2, dist1–dist2.
- **Correo:** P_emaildomain, R_emaildomain.
- **Conteos C1–C14:** cuántos elementos asociados (tarjetas, direcciones, correos) aparecen en la ventana de observación.
- **Deltas de tiempo D1–D15:** días transcurridos desde un evento previo (p.ej. primera transacción, cambio de tarjeta).
- **Coincidencias M1–M9:** variables booleanas de matching.
- **V1–V339:** variables propietarias de Vesta (rangos, conteos, hashes), con bloques fuertemente correlacionados.
- **Identidad id_01–id_38, DeviceType, DeviceInfo:** huella de dispositivo y de red.

2.2 Desbalance del problema

De las 590 540 transacciones de entrenamiento, 20 663 son fraudulentas: un 3,50 % (razón de desbalance 1:27, Fig. 2.1). Esta proporción hace que la exactitud (*accuracy*) sea una métrica inútil —un clasificador trivial que predice “legítima” alcanza 96,5 % de exactitud con utilidad nula—, por lo que la competencia adoptó el área bajo la curva ROC (AUC) como métrica oficial, que ordena transacciones en lugar de etiquetarlas con umbral fijo.

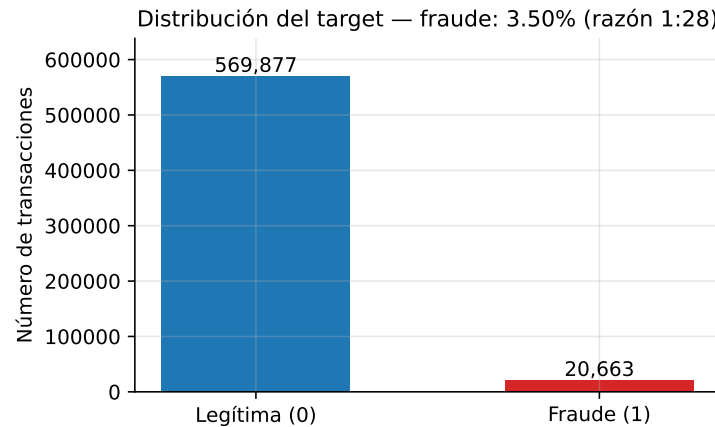


Figura 2.1: Distribución de la variable objetivo. El desbalance 1:27 exige métricas de orden (AUC) y hace innecesario el re-muestreo agresivo para modelos de árboles.

2.3 Calidad de los datos

Dos problemas dominan la calidad de los datos. Primero, los **valores faltantes**: la familia V promedia un 43 % de faltantes y la tabla de identidad, al fusionarse por completitud izquierda, introduce un 76 % de faltantes estructurales en sus columnas (Fig. 2.2). Segundo, la **cola pesada** del monto (Fig. 2.3), que en escala lineal concentra el 99 % de la masa por debajo de \$300 mientras la cola llega a \$31 937: los montos extremos son justamente sospechosos, por lo que no deben recortarse sino re-escalarse (logaritmo) o dejarse en bruto para los árboles.

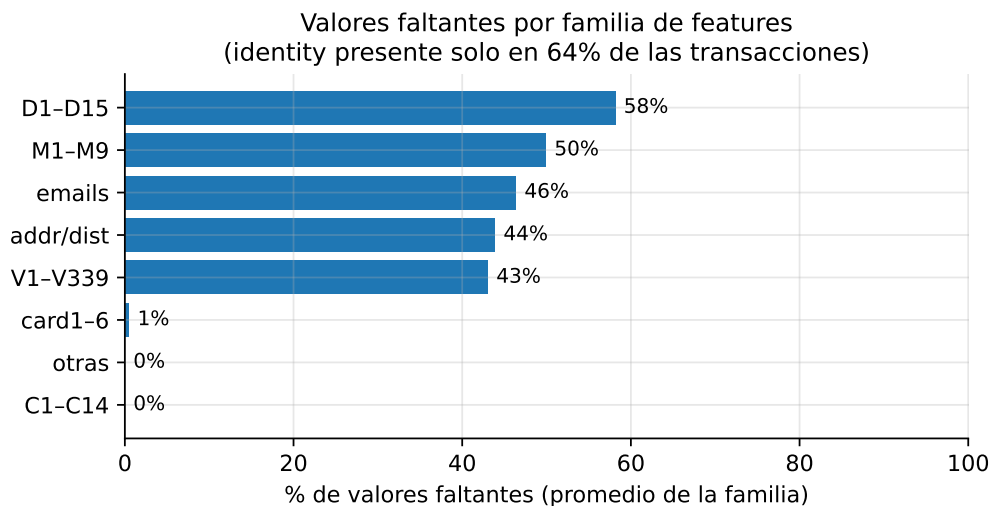


Figura 2.2: Porcentaje promedio de valores faltantes por familia de variables. La ausencia de la tabla de identidad es estructural (transacciones sin huella de dispositivo) y, como se verá, codificar la ausencia como categoría propia (-999) es preferible a imputar.

2.4 La estructura temporal: la dificultad central

TransactionDT es un desplazamiento relativo en segundos (693 999–1 581 113 131) que cubre 182 días de entrenamiento. La partición oficial de la competencia es **temporal**: el conjunto de prueba corresponde al mes siguiente al final del entrenamiento. Esto tiene tres consecuencias

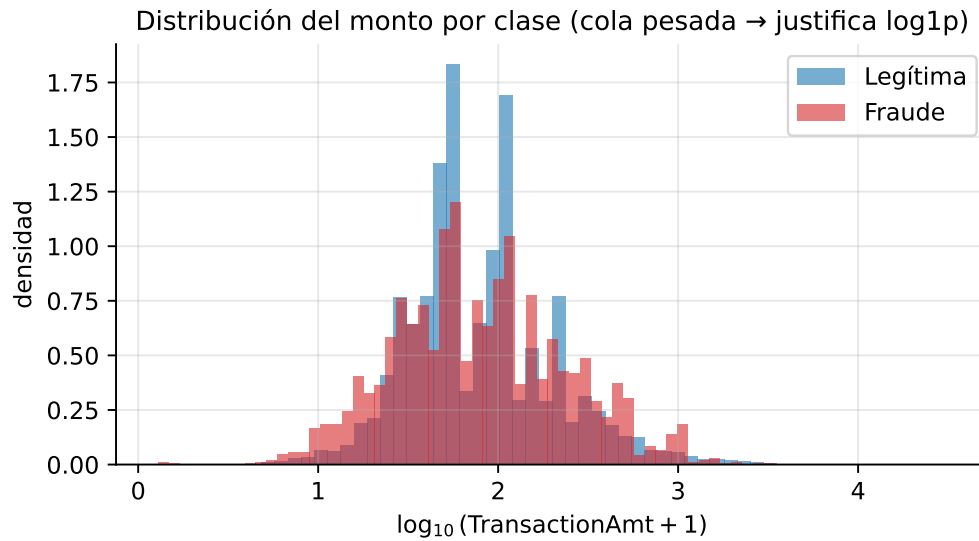


Figura 2.3: Distribución del monto por clase en escala logarítmica. Las transacciones fraudulentas se concentran en montos bajos-repetitivos, con una cola superior más densa que la de las legítimas.

metodológicas:

1. Una validación aleatoria (KFold clásico) *sobreestima* el desempeño: filas “del futuro” aparecen en el entrenamiento de cada fold.
2. Cualquier variable cuya distribución derive con el tiempo produce una relación espuria con el target (Fig. 2.4 y Section 3.4).
3. La validación honesta debe simular el futuro: *holdout* por corte temporal, `TimeSeriesSplit`, o `GroupKFold` agrupando por mes calendario.

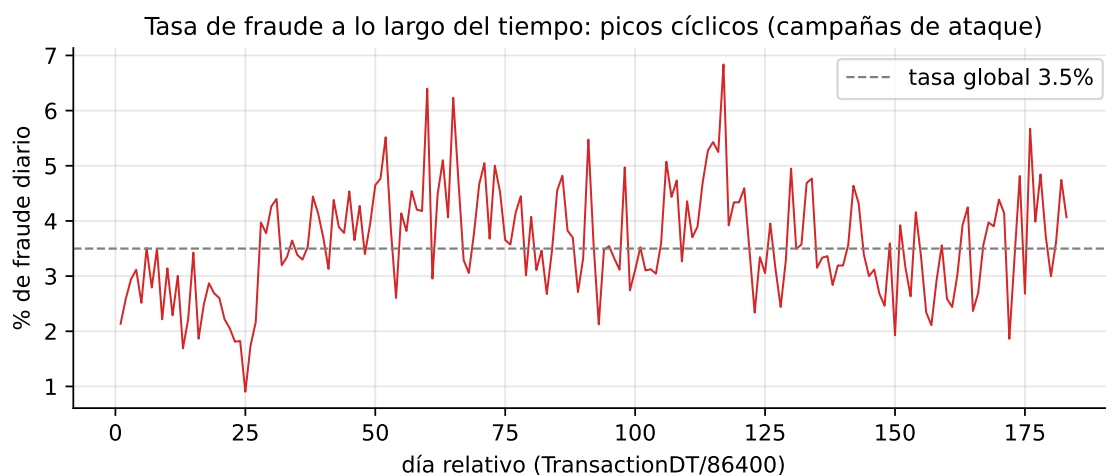


Figura 2.4: Tasa diaria de fraude durante los 182 días de entrenamiento. Los picos agudos corresponden a campañas de ataque acotadas en el tiempo: el patrón no es estacionario, y por eso la selección de variables por *consistencia temporal* (Section 3.5) resulta decisiva.

Como referencia de mercado, Fig. 2.5 muestra la distribución de puntajes del tablero público (6 355 equipos): la masa de competidores se concentra bajo 0,95 y el top-5 supera 0,966, lo que ilustra el diferencial que aportan las técnicas que este informe estudia.

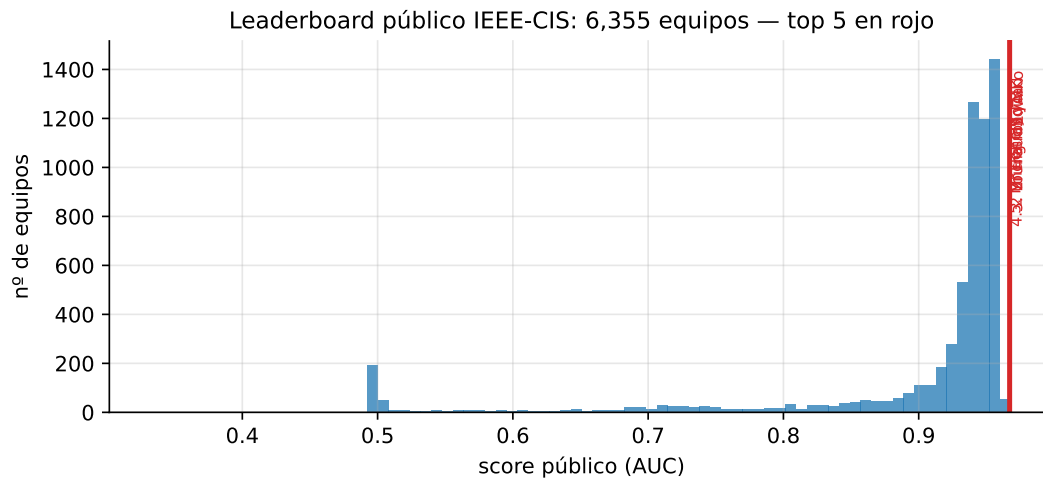


Figura 2.5: Histograma del tablero público de la competencia. En rojo, los cinco primeros equipos: AlKo (0,9681), FraudSquad (0,9677), Young for you (0,9676), 2 uncles and 3 puppies (0,9672) y Mr Lonely (0,9663).

Tratamiento de datos e ingeniería de variables

Este capítulo documenta las técnicas de preparación de datos extraídas del código real de los cuatro notebooks ejecutados, ordenadas de lo básico a lo que diferencia a la solución ganadora.

3.1 Integración y tipos de datos

Las dos tablas se integran por `TransactionID` con unión izquierda (la identidad es opcional). Dado que `train_transaction.csv` ocupa 2,2 GB en memoria por defecto, se aplican dos optimizaciones de tipo:

- numéricos a `float32` / `int16` donde el rango lo permite (reduce la memoria a la mitad, habilitando GPU);
- categóricas a `category` (diccionario de enteros) o `int16` vía `factorize`.

3.2 Valores faltantes: codificar en vez de imputar

Los notebooks de referencia convierten el faltante en *información*:

```
X_train = X_train.fillna(-999) # baseline (inversion, xhlulu)
clf = xgb.XGBClassifier(..., missing=-999)
```

El árbol de decisión aprende que `-999` es una “categoría desconocida” y puede ramificar sobre ella; imputar con la media, en cambio, fabrica valores plausibles y falsos. En la solución de Deotte el esquema es más fino: los numéricos se desplazan para hacerlos positivos y el faltante se codifica como `-1`:

```
mn = min(X_train[f].min(), X_test[f].min())
X_train[f] -= mn; X_train[f].fillna(-1, inplace=True)
```

3.3 Codificación de variables categóricas

Tres variantes aparecen en el código estudiado:

1. **Etiqueta entera** (`LabelEncoder` / `factorize`): suficiente para árboles, que tratan los códigos como categorías ordinales arbitrarias. Se ajusta sobre `train` y `test` juntos para compartir el diccionario.
2. **Codificación de frecuencia** (*count encoding*, `nroman`): reemplaza cada categoría por su frecuencia de aparición.

```
train['card1_count'] = train['card1'].map(train['card1'].value_counts(dropna=False))
```

Es una agregación “de bolsillo”: una tarjeta con miles de transacciones es una tarjeta comprometida, y esta señal queda expuesta como número.

3. **Interacciones como cadena** (`nroman`, `Deotte`): concatenar dos categóricas crea una entidad más fina.

```
train['card1__card5'] = train['card1'].astype(str) + '_' + train['card5'].astype(str)
train['uid'] = train['card1_addr1'].astype(str) + '_' + (day - D1).astype(str)
```

Variables derivadas simples pero efectivas: día de la semana y hora del día a partir de `TransactionDT`; parte decimal del monto ($(A - \lfloor A \rfloor) \times 1000$, patrón de centavos de bots); $\log_{10}$ del monto (kyakovlev).

3.4 Normalización de columnas D (estabilización temporal)

Las columnas D1–D15 significan “días transcurridos desde un evento previo” medidos *en el momento de la transacción*; por construcción derivan con el tiempo absoluto y el modelo confunde “mucho tiempo transcurrido” con “fraude”. La solución de Deotte resta el reloj absoluto:

```
for i in [4,5,6,8,10,11,12,13,14,15]:
    X_train['D'+str(i)] -= X_train.TransactionDT / (24*60*60)
    X_test ['D'+str(i)] -= X_test.TransactionDT / (24*60*60)
```

Fig. 3.1 muestra el efecto sobre D15: de una nube con pendiente (train y test viven en rangos distintos) a una banda horizontal estacionaria, directamente transferible al futuro. La misma idea es la que motiva la *validación adversarial* presente en el corpus estudiado: entrenar un clasificador train-vs-test y normalizar las variables que lo separan.

Normalización de columnas D (idea de Deotte): features comparables en el futuro

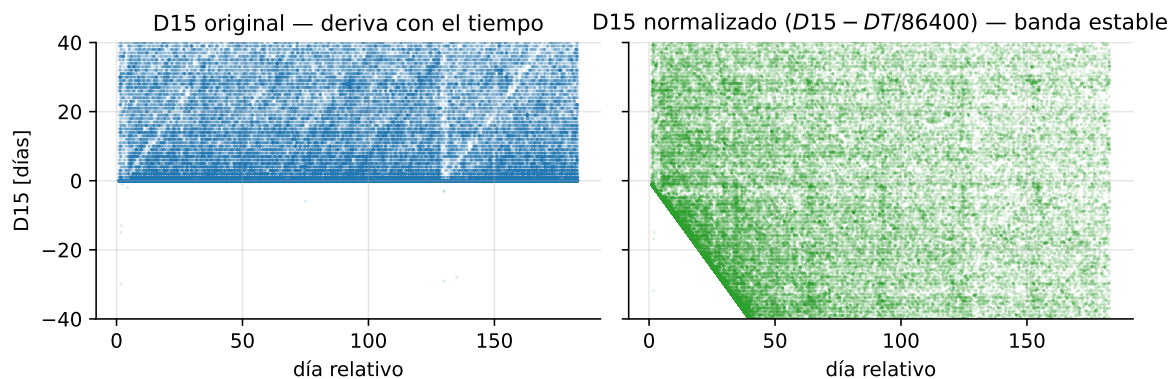


Figura 3.1: Normalización de D15. Izquierda: la variable cruda deriva con el tiempo. Derecha: tras restar el reloj absoluto queda una banda estable, comparable entre entrenamiento y futuro.

3.5 Selección por consistencia temporal

La solución de Deotte elimina explícitamente las variables cuyo histograma difiere entre el inicio y el fin del período de entrenamiento (p.ej. C3, M5, id_08, id_33, card4). El criterio es estadístico y barato: si una variable ya no es estable dentro del entrenamiento, su relación con el fraude tampoco lo será en el mes de prueba. De las 339 columnas `V` se conservan ~ 120 , elegidas por bloques de correlación en un EDA previo del mismo autor.

3.6 El pseudo-identificador de cliente (*magic UID*)

El dato que más ayudaría —el identificador del cliente— no existe en el conjunto. La solución ganadora lo reconstruye a partir de tres columnas:

$$\text{uid} = \underbrace{\text{card1_addr1}}_{\text{tarjeta + dirección}} \parallel \underbrace{\left\lfloor \frac{\text{TransactionDT}}{86400} - D1 \right\rfloor}_{\text{día estimado de alta de la cuenta}} \quad (3.1)$$

donde D1 (días desde la primera transacción) actúa como huella de antigüedad de la cuenta. La combinación sobrevive al cambio de dispositivo y de correo. Fig. 3.2 resume el esquema completo: a partir del UID se generan tres familias de agregaciones y *luego el UID se elimina*; nunca entra al modelo como variable.

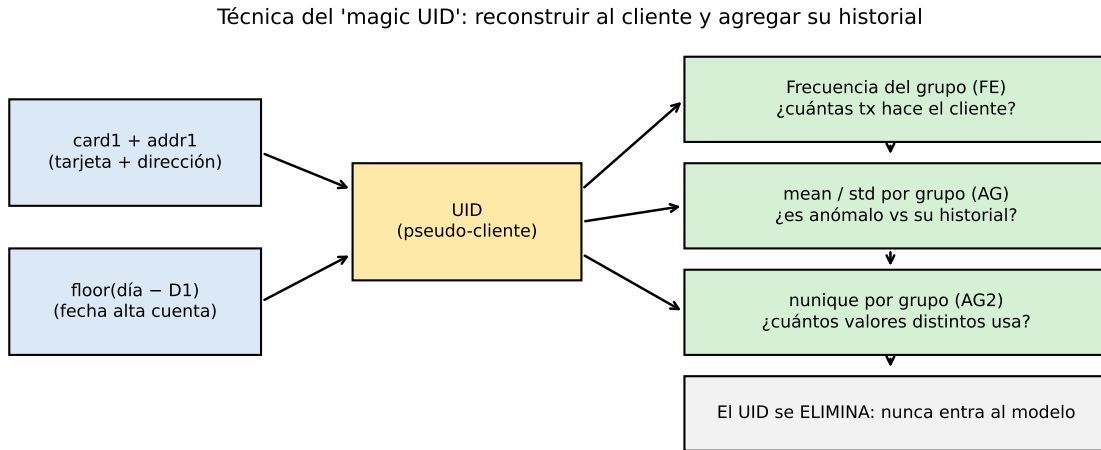


Figura 3.2: Esquema de la técnica del *magic UID*: reconstrucción del cliente a partir de (3.1) y generación de agregaciones por grupo. El identificador se usa solo como llave de agrupación y se descarta antes del entrenamiento.

Las tres familias de agregaciones, tal como aparecen en el código original:

```

# frecuencia del grupo: ¿cuántas transacciones acumula este cliente?
encode_FE(X_train, X_test, ['uid'])

# media y desviación por grupo: ¿es anómalo este monto/delta PARA ESTE cliente?
encode_AG(['TransactionAmt', 'D4', 'D9', 'D10', 'D15'], ['uid'], ['mean', 'std'])
encode_AG(['C'+str(x) for x in range(1,15) if x!=3], ['uid'], ['mean'])
encode_AG(['M'+str(x) for x in range(1,10)], ['uid'], ['mean'])

# cardinalidad por grupo: ¿cuántos valores distintos usa el cliente?
encode_AG2(['P_emaildomain', 'dist1', 'id_02', 'cents'], ['uid'])
encode_AG2(['C13', 'V314', 'V127', 'V136', 'V309', 'V307', 'V320'], ['uid'])

# interacción ad-hoc: dirección de envío de otro "mundo" temporal
X_train['outsider15'] = (np.abs(X_train.D1 - X_train.D15) > 3).astype('int8')
  
```

La intuición estadística es la reducción de varianza intra-grupo: un árbol que solo ve la variable X comete errores de contexto, mientras que la media de X dentro del grupo del cliente contextualiza cada observación y vuelve trivial la separación de los casos anómalos. Section 6.2 cuantifica este efecto en el experimento de ablación: +0,0104 de AUC con el modelo idéntico.

3.7 Protocolo de validación

Dada la partición temporal de la competencia, se adoptaron tres esquemas según el propósito:

Holdout temporal 75/25 entrenar con los primeros ~ 136 días y validar con los últimos ~ 46 . Rápido, para iterar (baseline, XGB_95/96).

TimeSeriesSplit $\times 3$ ventanas crecientes con validación siempre “hacia adelante” (nroman).

GroupKFold mensual $\times 3$ cada fold reserva *meses completos* como validación, agrupando por mes calendario reconstruido desde `START_DATE 2017-11-30` (XGB_95/96). Es el esquema más fiel al operativo real y el que se reporta como AUC local principal.

3.8 Limitación conocida del protocolo

Las agregaciones por UID se computan sobre *todo* el período de entrenamiento, incluido el mes reservado como validación (aunque *sin* usar la etiqueta del mes reservado para construir el modelo: el leak es de estadísticas de grupo, no de clases). Esto reproduce fielmente el protocolo original de la competencia y explica parte del nivel absoluto de AUC local; una variante estrictamente causal (agregar solo transacciones anteriores a cada `TransactionDT`) queda propuesta como trabajo futuro en [Section 8.2](#).

Sistema adaptativo y concept drift

El benchmark anterior sirve como línea base no adaptativa. Sobre él se implementó una evaluación por bloques temporales: cada bloque futuro se predice con las transacciones disponibles hasta ese momento, usando una ventana expansiva o una ventana deslizante de tamaño fijo. El protocolo permite observar la degradación del modelo, el efecto de olvidar historia y el costo de las decisiones.

4.1 Features causales

Se conserva la idea del *magic UID* del capítulo anterior, pero sus estadísticas se calculan de forma causal. Para una transacción en el instante t , la frecuencia y los promedios del UID usan solo observaciones anteriores a t . En el bloque de evaluación se emplean mapas construidos con el histórico previo; ninguna fila del futuro actualiza sus propias features antes de ser predicha. Esta corrección elimina la limitación transductiva identificada en [Section 7.3](#).

También se derivan el día relativo, la hora, el día de la semana, el mes calendario y versiones normalizadas de las columnas D . El identificador se elimina antes del entrenamiento: solo funciona como llave de agregación.

4.2 Ventanas y detección de drift

El experimento utiliza 60 días iniciales y bloques futuros de 30 días. Se comparan cuatro estrategias: histórico completo (ventana expansiva) y ventanas deslizantes de 30, 60 y 90 días. Para cada bloque se calcula el Population Stability Index (PSI) sobre monto, variables D y conteos C . Se marca drift distributivo cuando el PSI máximo supera 0,20; el rendimiento también se monitorea mediante AUC, PR-AUC, F1, recall, balanced accuracy y costo de decisión.

Sistema inteligente adaptativo para detección de fraude

El feedback de desempeño y drift decide cuándo actualizar el modelo

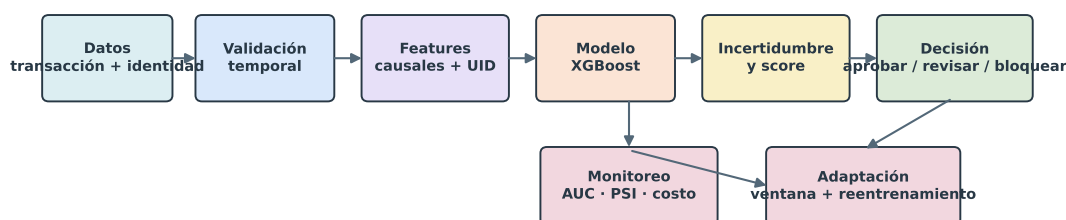


Figura 4.1: Arquitectura propuesta: el monitoreo temporal alimenta la selección de ventana y el reentrenamiento del modelo.

4.3 Resultados de la evaluación adaptativa

La corrida completa usa las 590 540 transacciones de entrenamiento, XGBoost con 180 árboles, umbral de decisión 0,20 y costos 1 para un falso positivo y 10 para un falso negativo. Se

reservan los bloques de los días 60–90, 90–120, 120–150 y 150–180 para evaluación futura.

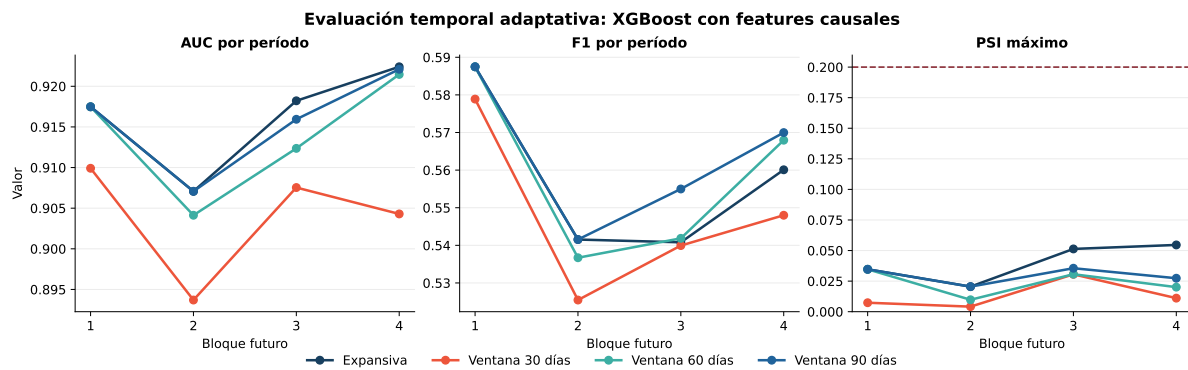


Figura 4.2: Evolución de AUC, F1 y PSI por bloque. El descenso del segundo bloque muestra que el rendimiento puede degradarse aunque el drift distributivo no supere el umbral alto de PSI; por ello se monitorean también métricas de desempeño.

La ventana expansiva obtuvo un AUC promedio de 0,9163 y un F1 promedio de 0,5575. Las ventanas de 60 y 90 días alcanzaron AUC promedio de 0,9139 y 0,9156, mientras que la ventana de 30 días obtuvo 0,9039. En esta corrida, conservar más historia funcionó mejor: la ventana corta perdió señal y no compensó el drift moderado observado. El AUC expansivo bajó de 0,9175 en el primer bloque a 0,9071 en el segundo, y el costo de decisión subió de 18059 a 21009 unidades. Por eso el sistema monitorea el desempeño aunque el PSI no active una alarma fuerte.

4.4 Decisión y adaptación operativa

El score se transforma en una acción con tres estados: aprobar, enviar a revisión o bloquear. El umbral puede modificarse según la utilidad del negocio. En producción, el sistema conservaría el modelo vigente mientras monitorea AUC y PSI. Ante una caída de rendimiento, un PSI alto o el cierre de un nuevo bloque temporal, recalcularía las features causales y reentrenaría con la ventana seleccionada. Todas las predicciones, decisiones y métricas deben quedar registradas para permitir auditoría humana y rollback del modelo.

Modelos comparados

Los cuatro notebooks ejecutables comparten la misma familia algorítmica —*gradient boosted trees*— y difieren en el tratamiento de datos, el hardware y el protocolo de validación. Esta homogeneidad es intencional: hace atribuible cualquier diferencia de AUC a la ingeniería de variables. [Table 5.1](#) resume las configuraciones.

Tabla 5.1: Configuraciones comparadas. “d9/d12” denota profundidad máxima del árbol.

	Baseline CPU	Baseline GPU	LightGBM (nroman)	XGB d12 (cdeotte)
Librería	XGBoost 3.4	XGBoost 3.4	LightGBM 4.x	XGBoost 3.4
Dispositivo	CPU (16 hilos)	RTX 2060	CPU (16 hilos)	RTX 2060
Profundidad	9	9	31 hojas	12
Árboles	500	500	≤5 000 (ES)	2 000–5 000 (ES)
Tasa de aprend.	0.05	0.05	0.0069	0.02
Features	432 crudas	432 crudas	298 (RFE + FE)	~200 (V + UID)
Catóricas	etiqueta	etiqueta	etiqueta	etiqueta + frecuencia
Validación	holdout 75/25	holdout 75/25	TSSplit ×3	GroupKFold mes ×3

5.1 Baseline XGBoost (inversion, xhlulu)

El punto de referencia: unión de tablas, relleno de faltantes con `-999`, etiquetado entero de catóricas y `XGBClassifier` de profundidad 9 con 500 árboles. La variante GPU delega el histograma y la construcción de árboles a la RTX 2060. En XGBoost moderno la invocación es:

```
xgb.XGBClassifier(n_estimators=500, max_depth=9, learning_rate=0.05,
                  subsample=0.9, colsample_bytree=0.9, missing=-999,
                  tree_method='hist', device='cuda') # ex 'gpu_hist'
```

5.2 LightGBM con selección de variables (nroman)

Parte de 120 variables “útiles” obtenidas por eliminación recursiva de características (RFE) en un notebook hermano, añade codificación de frecuencia, variables de calendario, interacciones y el decimal del monto, y entrena LightGBM con hiperparámetros de optimización bayesiana ([Table 5.1](#)), validación `TimeSeriesSplit` y parada temprana. El modelo final re-entrena con todo el período y el número óptimo de iteraciones.

5.3 XGBoost con ingeniería de entidades (cdeotte XGB_95 / XGB_96)

Un único diseño de modelo (profundidad 12, 2 000–5 000 árboles, tasa 0.02, muestreo 0.8/0.4) evaluado en dos niveles de tratamiento de datos:

XGB_95 (sin UID): selección de ~120 columnas V por bloques de correlación, normalización de columnas D, codificación de frecuencia de tarjeta/dirección/correo, combinaciones `card1+addr1(+email)`, agregaciones *por tarjeta* de monto y D9/D11, y descarte por consistencia temporal.

XGB_96 (con magic UID): todo lo anterior *más* el pseudo-cliente de [Section 3.6](#) y sus ~ 40 agregaciones. El modelo es idéntico al de XGB_95: la ablación de [Section 6.2](#) aísla el aporte del UID.

El notebook original cierra con un post-procesado que suaviza las probabilidades promediando dentro de cada UID (transacciones del mismo cliente deberían recibir scores similares); ese paso requiere listas externas de limpieza de UIDs y queda fuera del alcance local, documentado como trabajo futuro.

5.4 Costo computacional

La RTX 2060 (6 GB) acelera el entrenamiento $4.4\times$ respecto a 16 núcleos CPU en el baseline, con AUC indistinguible: la GPU es una palanca de *productividad* (más iteraciones por hora), no de precisión. El límite de memoria también define fronteras prácticas: el flujo *RAPIDS cuDF* del mismo autor (todo el preprocesamiento en GPU, $15\times$ más rápido) exige más de 6 GB de VRAM y no fue ejecutable en este hardware; queda documentado en el corpus descartado.

Resultados

6.1 Comparación principal

Table 6.1 presenta los resultados de la reproducción local. Todos los AUC provienen de validación temporal ejecutada en esta máquina sobre el conjunto completo de entrenamiento; no se realizó ninguna submisión a Kaggle (el estudio es estrictamente local).

Tabla 6.1: Ranking local: AUC y costo computacional por método. El AUC principal de cada método se reporta en su esquema de validación propio (columna Validación).

Modelo	AUC	Tiempo	Validación	Técnica clave
Baseline XGBoost CPU (inversion)	0.9191	112 s	holdout temporal 75/25	432 features crudas
Baseline XGBoost GPU (xhlulu)	0.9227	25 s	holdout temporal 75/25	identico al anterior, en GPU
LightGBM + FE (nroman)	0.9200	462 s	TimeSeriesSplit $\times 3$	RFE 120 features + count encoding
XGBoost d12 sin UID (cdeotte 95)	0.9375	163 s	GroupKFold mensual $\times 3$	V-cols seleccionadas + D normalizadas
XGBoost d12 + magic UID (cdeotte 96)	0.9479	192 s	GroupKFold mensual $\times 3$	UID + ~ 40 agregaciones por cliente

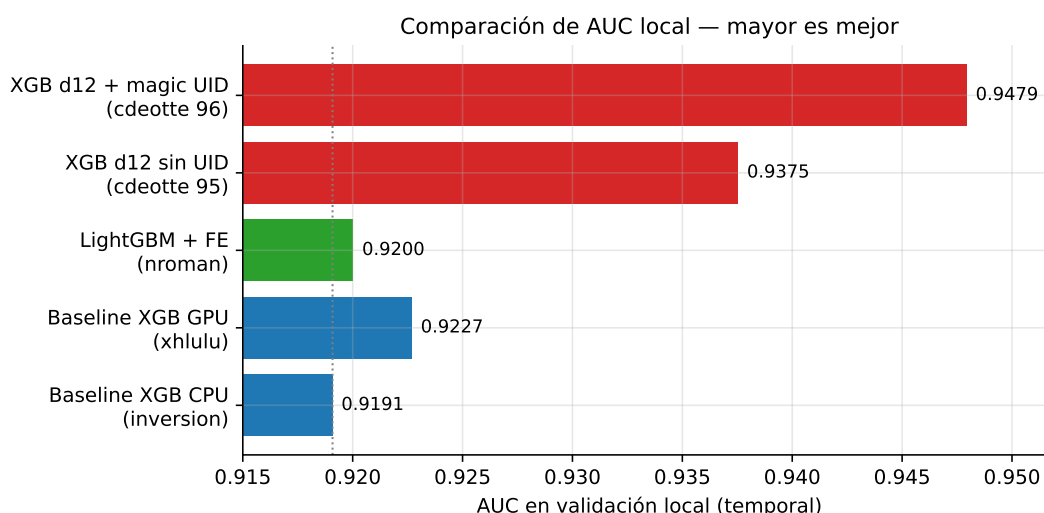


Figura 6.1: AUC local por método. El salto relevante ocurre al pasar de features crudas a la ingeniería de entidades (rojo): el mismo algoritmo gana +0,018 con la preparación de datos de Deotte y +0,010 adicionales con el magic UID.

6.2 Ablación del magic UID

Fig. 6.3 muestra el experimento clave. Con el modelo, los hiperparámetros y el hardware idénticos, añadir el pseudo-cliente y sus agregaciones produce:

$$\Delta AUC_{UID} = 0,9479 - 0,9375 = +0,0104 \quad (6.1)$$

medido con validación OOF GroupKFold mensual.

La ganancia es superior a la de cualquier otro cambio estudiado y se atribuye íntegramente a la representación de datos: el identificador nunca ingresa al árbol, solo sus estadísticas de grupo. En el holdout temporal 75/25 el efecto es consistente (0,9433 vs. 0,9370).

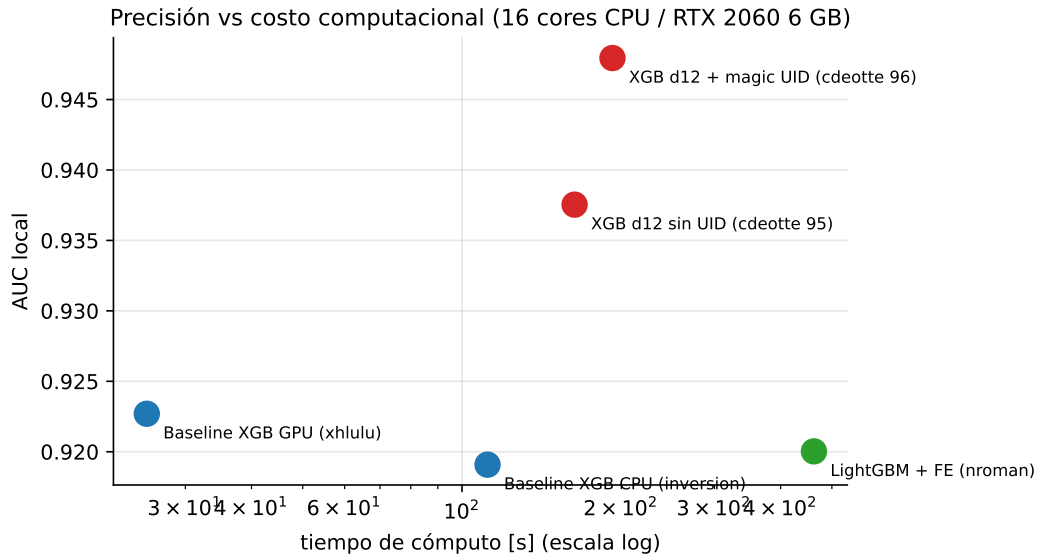


Figura 6.2: Precisión vs. costo computacional. La GPU reduce el tiempo del baseline de 112 s a 25 s (4.4×) sin alterar el AUC; la mayor precisión exige más cómputo (profundidad 12 y hasta 5 000 árboles) pero la ganancia dominante proviene de los datos, no del cómputo.

6.3 Qué variables dominan

Un ajuste exploratorio rápido de XGBoost (150 árboles, sobre las familias C/D/V y las columnas de tarjeta) confirma la estructura de señal esperada: dominan los conteos C13, C14, C1, los deltas de tiempo normalizables (D10, D15, D4) y las columnas de tarjeta/dirección que componen el UID (Fig. 6.4). Es coherente con la tesis del informe: la señal discriminante es *comportamental* (actividad por entidad y ventanas de tiempo), no transaccional aislada.

6.4 Referencias cruzadas con Kaggle

Los puntajes públicos de los mismos métodos en la competencia (fuera de este estudio, solo como referencia) fueron: baseline ~ 0.943 , nroman 0.9419, XGB_95 0.9514 y XGB_96 0.9627. El *ordenamiento* local coincide con el de la competencia: más ingeniería de entidades $\Rightarrow$ más AUC. El nivel absoluto local es menor porque el tablero evalúa el mes posterior con un cuarto de las transacciones y los puntajes públicos históricos se obtuvieron con submisión directa; nuestra validación local es deliberadamente más exigente.

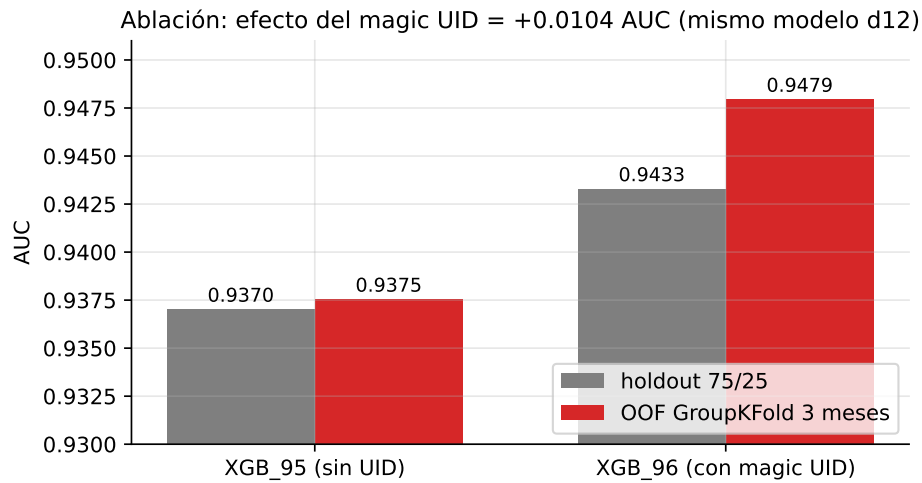


Figura 6.3: Ablación con/sin magic UID sobre el mismo modelo (XGBoost d12). La diferencia entre barras del mismo color es el aporte puro de la ingeniería de entidades.

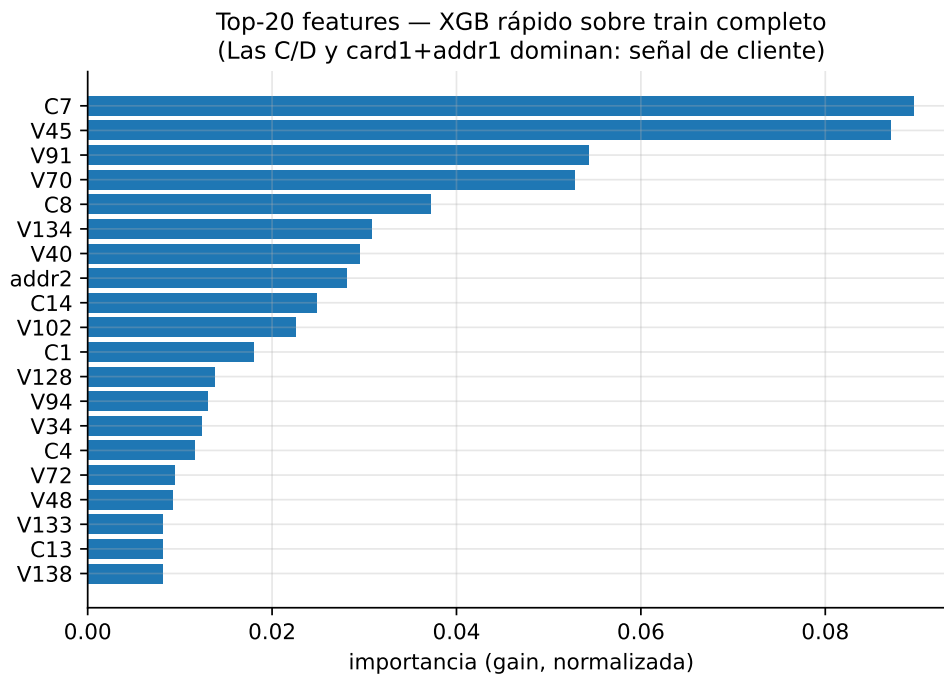


Figura 6.4: Importancia (gain) de las 20 variables principales en un XGBoost exploratorio. El peso de conteos, deltas temporales y tarjetas anticipa por qué las agregaciones por cliente resultan tan efectivas.

Discusión

7.1 Ventajas y límites de cada método

Tabla 7.1: Ventajas y limitaciones observadas en la reproducción local.

Método	Ventaja principal	Límite principal
Baseline XGBoost CPU	Mínimo costo de desarrollo; piso de referencia inmediato (~ 0.92).	Features crudas desperdician la señal de entidad; 112s por ajuste frena la iteración.
Baseline XGBoost GPU	4.4× más rápido con AUC idéntico; habilita búsqueda de hiperparámetros.	No aporta precisión por sí solo; exige VRAM suficiente.
LightGBM + RFE + count encoding (nroman)	30 % menos features con AUC equivalente; validación temporal estricta; interpretable.	Ganancia de precisión marginal; la tasa de aprendizaje 0.0069 exige miles de árboles.
XGBoost d12 + magic UID (cdeotte)	Mayor AUC local (0.9479); ganancia atribuible y replicable (+0,0104).	Requiere conocimiento del dominio (composición del UID); agregaciones costosas; protocolo transductivo (ver Section 7.3).

7.2 Qué importa realmente: jerarquía de impacto

Los experimentos permiten ordenar los factores por su contribución al AUC:

1. **Validación honesta** (no aporta AUC, evita ilusiones): el esquema de validación decide si un resultado es real. KFold aleatorio habría reportado valores notablemente superiores e irrepetibles en producción.
2. **Ingeniería de entidades** (+0,028 acumulado): el pseudo-cliente y sus agregaciones explican la mayor parte de la distancia entre el baseline y el tope local. La lección es transferible a cualquier dominio con entidades implícitas (usuarios, dispositivos, comercios): *construir la entidad, agregar su historial, descartar la llave*.
3. **Estabilización temporal** (habilitadora): normalizar columnas D y filtrar por consistencia temporal no sube el AUC por sí sola, pero hace que las ganancias anteriores *se transfieran* al futuro.
4. **Selección de variables** ($\approx$ neutral aquí): recortar de 432 a 120 variables mantuvo el AUC con menos costo, relevante en producción.
5. **Algoritmo e hiperparámetros** (menor): entre XGBoost y LightGBM la diferencia observada es inferior al efecto de cualquier técnica de datos; la optimización bayesiana de nroman vale menos que una sola feature de entidad bien construida.
6. **Hardware** (0 AUC, $\times 4.4$ velocidad): la GPU acelera el ciclo de experimentación, que es donde se generan las ganancias de datos.

7.3 Limitaciones metodológicas

1. **Protocolo transductivo:** las agregaciones por UID se computan sobre todo el período de entrenamiento, incluidos los meses usados como validación (sin usar sus etiquetas). El AUC local absoluto es, por tanto, un límite superior respecto de un sistema causal estricto. La variante estricta (estadísticas solo con transacciones anteriores a cada transacción) es el siguiente hito propuesto.
2. **Un solo grano de validación para los XGB d12:** GroupKFold de 3 meses (6 en el original) por restricciones de tiempo de GPU; las diferencias entre folds fueron pequeñas y consistentes en dirección.
3. **Corpus no ejecutado:** 8 notebooks de alto valor analítico (EDA, re-muestreo, optimización bayesiana completa, *blend* de artgor, RAPIDS de Deotte) se analizaron cualitativamente pero no se reprodujeron numéricamente; sus conclusiones no cuentan con medición local.
4. **Métrica única:** el AUC ordena bien pero no responde por costos asimétricos de negocio (falso positivo bloquea un pago legítimo); un despliegue real requiere umbral y métrica de utilidad propios.

7.4 Amenazas a la validez

Las adaptaciones de librerías (XGBoost ≥ 2 , LightGBM ≥ 4 , pandas 3) alteran detalles de desempate y de parada temprana respecto de 2019; se priorizó preservar la *lógica* de cada pipeline sobre la igualdad bit a bit. Las diferencias con los puntajes históricos de Kaggle son esperadas y se documentan en [Chapter 6](#); ninguna cambia la conclusión comparativa.

Conclusiones y propuesta

8.1 Conclusiones

1. La reproducción local de las mejores soluciones públicas de IEEE-CIS Fraud Detection es viable en hardware de consumidor (16 núcleos, RTX 2060 de 6 GB) y entrega resultados cualitativamente idénticos a los de la competencia: el ordenamiento por AUC se conserva.
2. El experimento de ablación demuestra que el rendimiento excepcional de la solución ganadora no proviene del algoritmo (idéntico en ambas variantes) ni del hardware, sino de la **ingeniería de variables orientada a entidades**: reconstruir el cliente implícito (*magic UID*) y agregar su historial aporta +0,0104 de AUC local, más que cualquier cambio de modelo estudiado.
3. La **estabilización temporal** de variables (normalización de columnas D, filtrado por consistencia temporal) y la **validación temporal honesta** (GroupKFold mensual) son condiciones habilitantes: sin ellas, las ganancias no se transferirían al mes de prueba.
4. El hardware acelera 4.4× el ciclo de experimentación sin mover el AUC: su valor es de productividad, no de precisión.
5. Las técnicas de re-muestreo (SMOTE) presentes en el corpus analítico no forman parte de las soluciones ganadoras: con árboles y AUC, el desbalance se maneja vía objetivos probabilísticos y `scale_pos_weight`.

8.2 Propuesta de desarrollo

Sobre la evidencia recogida, la propuesta consiste en construir el pipeline siguiente, en orden de retorno esperado:

1. **Base**: XGBoost/LightGBM con las familias C/D/V seleccionadas por correlación y columnas D normalizadas (componentes ya verificados).
2. **Entidades causales**: implementar el magic UID con agregaciones *estrictamente históricas* (cada transacción usa solo estadísticas de transacciones anteriores a su `TransactionDT`), eliminando el carácter transductivo detectado en el protocolo original.
3. **Ensemble**: combinar por promedio de rangos el XGB de entidades con un LightGBM equivalente, siguiendo el camino del equipo ganador (0,9459 en el tablero privado), y añadir el post-procesado de suavizado por UID.
4. **Evaluación de negocio**: incorporar umbral de decisión y curva de utilidad (costo de bloqueo vs. pérdida por fraude) junto al AUC.

Con ello, el proyecto transita de *reproducir* las mejores técnicas públicas a *producir* una solución propia, superando la limitación metodológica principal detectada.

Reproducibilidad

A.1 Estructura del proyecto

```

PROYECTO1_PLANIFICA/
|-- data/raw/                dataset oficial (CSV) + leaderboard publico
|-- codigos/
|   |-- ejecutados/         4 notebooks reproducidos (.ipynb + .py)
|   |-- descartados/        8 notebooks analizados sin ejecutar
|   `-- _RESUMEN_TECNICAS.md inventario de tecnicas por notebook
|-- src/                     benchmarks reproducibles
|   |-- bench_01_inversion.py baseline XGB CPU
|   |-- bench_02_xhlulu.py   baseline XGB GPU
|   |-- bench_03_nroman.py   LightGBM + FE (TimeSeriesSplit)
|   |-- bench_04_cdeotte.py  XGB d12 con/sin magic UID
|   |-- download_notebooks.py descarga via API publica de Kaggle
|   |-- extract_notebooks.py exportacion .ipynb -> .py
|   `-- make_ranking.py      consolidacion de resultados
|-- results/                 CSV y logs de cada benchmark + ranking
`-- informe_latex/           este informe (tectonic main.tex)

```

A.2 Entorno

- Linux; Python 3.12 en entorno uv; pandas 3.0, XGBoost 3.4, LightGBM 4.x, scikit-learn.
- CPU 16 núcleos; NVIDIA RTX 2060 6 GB (driver 610).
- Datos: `kaggle competitions download -c iieee-fraud-detection` (requiere aceptar las reglas de la competencia).

A.3 Ejecución de los experimentos

```

uv venv --python 3.12 .venv
uv pip install pandas numpy scikit-learn lightgbm xgboost
uv pip install matplotlib
.venv/bin/python src/bench_01_inversion.py      # AUC 0.9191, 112 s
.venv/bin/python src/bench_02_xhlulu.py         # AUC 0.9227, 25 s (GPU)
.venv/bin/python src/bench_03_nroman.py         # AUC 0.9200, 462 s
.venv/bin/python src/bench_04_cdeotte.py        # 0.9375 -> 0.9479 (GPU)
.venv/bin/python informe_latex/scripts/generar_graficos.py
tectonic informe_latex/main.tex

```

A.4 Adaptaciones de versiones (2019 → 2026)

- **XGBoost ≥2**: `gpu_hist` se reemplaza por `hist` con dispositivo `cuda`; los parámetros de parada temprana pasan al estimador.
- **LightGBM ≥4**: los argumentos de parada temprana y `verbose_eval` de `lgb.train` se sustituyen por `callbacks`.

- **pandas 3:** las cadenas leídas de CSV adoptan el dtype `str` (no `object`); las condiciones `dtype=='object'` deben reemplazarse por `pd.api.types.is_numeric_dtype(...)` negado. `LabelEncoder` se sustituye de forma segura por `pd.factorize(sort=True)` cuando hay tipos mixtos.
- **RAPIDS cuDF** (notebook GPU de Deotte): requiere >6 GB de VRAM; no ejecutable en la RTX 2060, se analizó cualitativamente.

A.5 Protocolo ético

El estudio se limita a material público con licencia abierta (Apache 2.0 en los notebooks citados). No se realizó ninguna submisión a la competencia ni uso de servicios externos con los datos; toda la evidencia numérica proviene de ejecuciones locales.

Bibliografía

- [1] Jihoo Kim (datartist). Must-see Kernels for IEEE-CIS Fraud Detection — índice curado. <https://www.kaggle.com/competitions/ieee-fraud-detection/discussion/104973>, 2019. 165 votos; reputaciones y categorías de los mejores notebooks.
- [2] IEEE Computational Intelligence Society and Vesta Corporation. IEEE-CIS Fraud Detection — kaggle competition. <https://www.kaggle.com/competitions/ieee-fraud-detection>, 2019. 6355 equipos; métrica oficial: AUC.